

Comparative Analysis Report

1. Structural comparison

How does the search space of the 8-puzzle compare structurally to the game tree of Tic-Tac-Toe? What is similar, and what is fundamentally different? (Hint: consider who controls the next state, and what "solving" means in each case.)

- The search space of the 8-puzzle and the game tree of Tic-Tac-Toe both consist of a collection of states of a 3x3 board, represented as a 2d array.
- The next state in the 8-puzzle is controlled by one agent which is either the AI or a human all throughout the game. In Tic-Tac-Toe, two agents are playing the game, and the decision of one agent affects the state for the next agent. In the latter game, the agents are turn-taking and can be human vs human, AI vs AI, or human vs AI.
- The 8-puzzle has a defined goal that is always the same ([1,2,3],[4,5,6],[7,8,9]); while Tic-Tac-Toe has multiple possible goal states, such as 3 diagonals, 3 horizontals, or 3 verticals. Tic-Tac-Toe does not guarantee to reach any of these states and can also end in a draw.

2. Algorithm fit

Why is A appropriate for the 8-puzzle but not directly applicable to Tic-Tac-Toe? Conversely, why does Minimax not naturally apply to the 8-puzzle?*

- A* is appropriate for the 8-puzzle because it can guarantee to find the shortest path to the goal state with the appropriate heuristics, which is exactly what the 8-puzzle needs to order the tiles. This algorithm is not directly applicable to Tic-Tac-Toe because it cannot consider the other player's moves and how they can affect reaching the final goal.
- Minimax is designed for two players in mind because it has max which is optimizing its own score, and min which is trying to minimize max's score. This naturally applies to Tic-Tac-Toe because it is a two-player zero-sum game. The 8-puzzle is a one player game making Minimax unnecessary and inefficient for it.

3. Empirical comparison — Module A

Report nodes expanded for blind search, Dijkstra, and A on the standardized test puzzle. What does this tell you about the value of heuristic information?*

Start state: $[[8,1,3], [4,0,2], [7,6,5]]$

AI Search Lab

ACTIVE MODULE

Search Algorithms

8-Puzzle Solver

Tic-Tac-Toe AI

PERFORMANCE

ALGORITHM

BFS

DECISION TIME

0.00 ms

NODES EXPLORED

0

SOLUTION LENGTH

-

8-Puzzle Solver

Use Image Use Numbers

8	1	3
4		2
7	6	5

BFS Solve << >> Shuffle

Import State: No file selected

- **Blind Search: BFS**

AI Search Lab

ACTIVE MODULE

Search Algorithms

8-Puzzle Solver

Tic-Tac-Toe AI

PERFORMANCE

ALGORITHM

BFS

DECISION TIME

20.00 ms

NODES EXPLORED

5124

SOLUTION LENGTH

14

8-Puzzle Solver

Use Image Use Numbers

1	2	3
4	5	6
7	8	

BFS Solve << >> Shuffle

Import State: No file selected

- Dijkstra

AI Search Lab

ACTIVE MODULE

Search Algorithms

8-Puzzle Solver

Tic-Tac-Toe AI

PERFORMANCE

ALGORITHM

Dijkstra

DECISION TIME

164.00 ms

NODES EXPLORED

5124

SOLUTION LENGTH

14

8-Puzzle Solver

Use Image Use Numbers

1	2	3
4	5	6
7	8	

Dijkstra

Solve

◀ ▶ ▶▶

Shuffle

Import State:

No file selected

- A* (Manhattan Distance)

AI Search Lab

ACTIVE MODULE

Search Algorithms

8-Puzzle Solver

Tic-Tac-Toe AI

PERFORMANCE

ALGORITHM

A* (Manhattan)

DECISION TIME

2.00 ms

NODES EXPLORED

77

SOLUTION LENGTH

14

8-Puzzle Solver

Use Image Use Numbers

1	2	3
4	5	6
7	8	

A* (Manhattan)

Solve

◀ ▶ ▶▶

Shuffle

Import State:

No file selected

Algorithm	Decision Time	Nodes Explored	Solution Length
BFS	20ms	5124	14
Dijkstra's	164ms	5124	14
A* (Manhattan)	2ms	77	14

All search algorithms found the optimal solution depth of 14 moves from $[[8,1,3],[4,0,2],[7,6,5]]$ to $[[1,2,3],[4,5,6],[7,8,0]]$. While BFS and Dijkstra's both explored the same number of nodes, I was surprised by the significant time difference between both algorithms. Initially, I assumed BFS and Dijkstra's would share the same results across the three metrics. However, it makes sense that Dijkstra's would take more time because it is not only exploring the nodes, but it is keeping track and sorting the nodes with a priority queue. A* only explores 77 nodes which means the heuristic chosen is appropriate for the task.

4. Empirical comparison — Module B

Report nodes explored by Minimax vs Alpha-Beta on the standardized test position. Compute and report the observed pruning rate.

Start state: empty board

AI Search Lab

ACTIVE MODULE

Search Algorithms

8-Puzzle Solver

Tic-Tac-Toe AI

PERFORMANCE

Player X (-)

DECISION TIME

0.00 ms

NODES EXPLORED

0

PRUNING EFFICIENCY

Player O (-)

DECISION TIME

0.00 ms

NODES EXPLORED

Tic-Tac-Toe with AI

Real-time visualization of adversarial search algorithms

Mode:

AI vs AI

First Player:

X starts

AI (X):

Minimax

AI (O):

Alpha-Beta

Start Game

- AI with Minimax plays first as X

AI Search Lab

ACTIVE MODULE
Search Algorithms

8-Puzzle Solver

Tic-Tac-Toe AI

PERFORMANCE

Player X (Minimax)

DECISION TIME
52.00 ms

NODES EXPLORED
549945

PRUNING EFFICIENCY
0.00%

Player O (-)

DECISION TIME
0.00 ms

NODES EXPLORED

Tic-Tac-Toe with AI

Real-time visualization of adversarial search algorithms

X

O's turn

Mode:
AI vs AI

First Player:
X starts

AI (X):
Minimax

AI (O):
Alpha-Beta

Restart Game

- AI with Alpha-Beta plays first as X

AI Search Lab

ACTIVE MODULE
Search Algorithms

8-Puzzle Solver

Tic-Tac-Toe AI

PERFORMANCE

Player X (Alpha-Beta)

DECISION TIME
10.00 ms

NODES EXPLORED
34202

PRUNING EFFICIENCY
93.78%

Player O (-)

DECISION TIME
0.00 ms

NODES EXPLORED

Tic-Tac-Toe with AI

Real-time visualization of adversarial search algorithms

O's turn

Mode:
AI vs AI

First Player:
X starts

AI (X):
Alpha-Beta

AI (O):
Minimax

Restart Game

Algorithm	Decision Time	Nodes Explored	Pruning Rate
Minimax	52ms	549945	N/A
Alpha-Beta	10ms	34202	93.78%

Alpha-Beta pruning explored significantly less nodes than Minimax, and it took $\frac{1}{4}$ of Minimax's time.

5. Trade-off analysis

Briefly discuss completeness, optimality, time complexity, and space complexity for each algorithm. A one-sentence summary per property per algorithm is sufficient.

BFS is complete because it guarantees to find a solution after traversing level-by-level, and it is optimal in unweighted graphs where it can find the shortest path solution. In the 8-puzzle, BFS had to explore 5124 nodes until reaching the solution. Its time complexity is $O(|V| + |E|)$ and it has a high space complexity of $O(V)$ because it stores all nodes at each level.

Dijkstra's is complete and optimal for nonnegative weights where it can find the lowest cost path, but in the 8-puzzle it is not relevant since each state of the board will have the same weight. Therefore, Dijkstra's will explore a similar number of nodes for the 8-puzzle and share a similar space complexity to BFS $O(V)$. The time taken by Dijkstra's (164ms) is higher than BFS's (20ms) because it has to sort the nodes in a priority queue which increases the time complexity to $O((|V| + |E|) \log |V|)$.

While Dijkstra's uses only the past cost to find the best solution, A^* uses both past and future cost and it is complete and optimal with an admissible heuristic. Since moves in the 8-puzzle are only horizontal or vertical, Manhattan Distance is used as the heuristic. A^* has a $O((|V| + |E|) \log |V|)$ time complexity and $O(V)$ space complexity, but in practice this is better. In the 8-puzzle, A^* only had to expand 77 nodes and took (2ms).

Minimax is a complete and exhaustive algorithm that explores all nodes to find the optimal solution, in the first move of Tic-Tac-Toe it explored 549945 nodes, but due to this it may not be efficient in large state spaces. Its time complexity is $O(b^m)$ with b being the branch factor and m the tree depth and $O(m)$ space complexity.

Alpha-Beta pruning optimizes minimax because it prunes nodes that do not influence the final decision. As seen in Tic-Tac-Toe, Alpha-Beta only explored 34202 nodes to select a good starting position for X, making the algorithm both complete and optimal. Its time complexity can improve from minimax depending on the node ordering, and it also has an $O(m)$ space complexity.

Heuristic justification

Manhattan distance is used for the 8-puzzle because the possible moves are either horizontal or vertical (4 directions). It is admissible for the 8-puzzle because it counts the minimum number of moves each tile needs to reach the goal position for the solution. The estimated cost is less than the actual cost because each move consists of moving one tile at a time, so the distance is reduced at most by 1. The heuristic does not overestimate the actual distance, making it admissible and consistent for A* search.